

Edson David Garcia Huanca

Entrypoints

```
"start": "NODE_ENV=production node index.js",  
"test": "NODE_ENV=test node --test",  
"dev": "NODE_ENV=development ./bin/nodemon index.js"
```

start: la BDD se guarda en su sitio, como ahora tampoco hay BDD especifica, esta se carga en la RAM igualmente

test: la BDD se guarda en la RAM

dev: mismo que test, simplemente que hace eso de cuando se modifica un archivo se reinicia la ejecucion

indicant la ruta, el metode, quines dades espera (si ´es el cas), quina acci´o fa a l'aplicaci´o, si requereix autenticaci´o, i quins rols s´on necessaris (si ´es el cas)

```
app.use("/api/tournaments", tournamentsRouter);  
app.use("/api/users", usersRouter);  
app.use("/api/login", loginRouter);  
app.use("/api/videogames", videogamesRouter);
```

Si no es diu res, no es requereix role

```
/api/login
```

post("/")

Solicita: username, password

Devuelve: token firmado si los datos coinciden con un usuario

Comprueba password usando bcrypt

```
/api/users
```

get("/")

Solicita: Nada

Devuelve: Lista de usuarios

get("/:id")

Solicita: Nada

Devuelve: Info del usuario (menos la contrasenya), si no existe devuelve err 404

post("/createUser")

Solicita: username, password, email

Devuelve: Info del usuario creado correctamente

Comprueba que el usuario y el email son unicos, si no lo son devuelve error

post("/createSpecialUser")

Solicita: username, password, email, role

Devuelve: Info del usuario creado correctamente

Comprueba que el usuario y el email son unicos, si no lo son devuelve error

Comprueba que el solicitante es un admin

`/api/tournaments`

`get("/")`

Solicita: Nada

Devuelve: Lista de torneos

`get("/:id")`

Solicita: Nada

Devuelve: Info del torneo, si no existe devuelve err 404

`get('/:id/participants')`

Solicita: Nada

Devuelve: Info de los participantes del torneo, si no existe devuelve err 404

`get('/:id/games')`

Solicita: Nada

Devuelve: Info de los games del torneo, si no existe devuelve err 404

`get('/:id/games/:gameId')`

Solicita: Nada

Devuelve: Info de un game del torneo

si no existe o no es del torneo especificado devuelve err 404

`post("/")`

Solicita: name, description, videogame, type, rounds, tournament_start_date, tournament_end_date, token

Devuelve: Info del torneo creado correctamente o error

Comprueba que solicitante sea un organizer mediante el token

Comprueba que se han pasado todos los datos necesarios

Comprueba que el nombre sea unico

Comprueba que el juego este en la lista de juegos

Crea los juegos del torneo segun la ronda

`delete("/:id")`

Solicita: token

Devuelve: Nada

Comprueba que solicitante sea un organizer mediante el token

Comprueba que el id corresponde a un torneo

Elimina el torneo

`patch("/:id")`

Solicita: Informacion a cambiar y token

Devuelve: Mensaje

Comprueba que el torneo exista

Comprueba que el token corresponde al dueño del torneo

Comprueba que la solicitud tenga almenos algo

Modifica el Torneo, cambiando los valores cambiados

(No modifica ni el nombre ni la ID)

patch("/:id/games/:gameId")

Solicita: Informacion a cambiar y token

Devuelve: Mensaje

Comprueba que el torneo exista

Comprueba que el game exista

Comprueba que el game sea del torneo

Comprueba que el token corresponde al dueño del torneo

Comprueba que la solicitud tenga almenos algo

Modifica el game, cambiando los valores cambiados

(No modifica ni la ronda ni la ID)

post("/:id/register")

Solicita: token

Devuelve: Mensaje

Comprueba que el id corresponde a un torneo

Comprueba que solicitante sea un user mediante el token

Comprueba que el user no se registre 2 veces

Registra al user al torneo

delete("/:id/participants/:userId")

Solicita: token

Devuelve: Mensaje

Comprueba que el id corresponde a un torneo

Comprueba que el userId corresponde a un usuario inscrito al torneo

Comprueba que solicitante sea el user inscrito o el organizador del torneo mediante el token

deshace el registro al user del torneo

/api/videogames

get("/")

Solicita: Nada

Devuelve: Lista de juegos

get("/:id")

Solicita: Nada

Devuelve: Info del juego, si no existe devuelve err 404

post("/")

Solicita: name, description, videogame, type, tournament_start_date, tournament_end_date, token

Devuelve: Info del juego creado correctamente o error

Comprueba que solicitante sea un admin mediante el token

Comprueba que se han pasado todos los datos necesarios

Comprueba que el nombre sea único

delete("/:id")

Solicita: token

Devuelve: Nada

Comprueba que solicitante sea un admin mediante el token

Comprueba que el id existe a un torneo

Elimina el juego

Les comandes SQL a models, són molt flexibles, les restriccions estan principalment als controllers.

Justificació base de dades

users: degut a que no tenia motius per afegir-ne molta informació he decidit deixar-ho simple

tournaments: he decidit al final fer-ho com ho he fet, volia canviar algunes coses com que stateRegistration sigui una data/String, però complicava molt les coses, també he tingut problemes en com fer per crear els games en crear un torneig, al final simplement he fet que només accepti certs valors de "type" i aquests valors tenen un codi que crea automàticament als games segons el número de "rounds" donat

Per temes de temps, i que no ho he considerat necessari, només he usat "torneig", ja que implementar més simplement seria afegir lògica de quants games es creen.

tournament_registrations: una manera de tindre una taula per controlar els registres als torneigs de part dels usuaris, aplicaria la mateixa taula/lògica per controlar els organitzadors extres que pot afegir els organitzadors que només poden modificar els games

videogames: molt simple, no he volgut complicar-me, podria afegir puntuació i més coses, però com cada joc pot dependre de cada cosa i no em servia de molt complicar-ho sense molt sentit he ficat el menys possible

games: seguint la lògica dels registres he fet els games, també aquí he afegit això d'ON DELETE CASCADE per tal que quan s'elimini el torneig, també s'eliminïn els games i els registres, en aquest cas state ho he afegit perquè m'aportava claredat

TESTS

Principalment, m'he assegurat de què funcioni el codi, hi ha moltes excepcions que no he comprovat si funcionen, i hi ha casos en els quals dono per suposat que alguna cosa funciona.

per exemple: després d'assegurar-me que el login_api.test funcione bé, feia copiar i pegar del login en els següents testos

Després de molt temps, he modificat els testos, ja que he afegit un helper.js perquè hi havien coses que feia moltes vegades i el codi es feia molt llarg i una mica tediós d'analitzar, també quan canviava alguna cosa havia de modificar moltes coses, així que he decidit fer-ho més llegible.

Al principi no tenia molta idea de com fer els tests, i quan tenia molta part feta, va ocórrer de simplificar moltíssima part usen funcions, ja que com he volgut assegurar-me tot el que podia de què funcionava, feia moltes vegades login i això ocupava moltes línies. A més que podia separar els cossos segons el rol.